

LZSCC.461 – Programming for Data Scientists

Introduction to R

Dr. Stefan Mühlbauer

Week 0

Your first week for LZSCC.461

1. **Today's lab:** I will give

- an overview of the whole module
- set-up instructions for R & RStudio
- an intro to R
- an intro to R Markdown
- reproducibility & writing coursework

2. **You** will go through Chapter 0 of the lecture notes



Students not in LZSCC: please use this link: <https://smba.github.io/r-programming>.

Overview

Structure of the module

- **Weeks 1–5: R programming**
 - Focus on the statistical side of programming
 - 2-hr lecture: overview of the week, live coding, feedback on the past week
 - 2-hr in-person lab each week
 - Quiz at the start of each lab on the past week (not graded, don't worry!)
 - Then you work through the weekly notes and solve exercises
- Weeks 6–10: Python

An overview of the R part

- Chapter 0: Intro to R (this slide deck)
- Chapter 1: Data manipulation and plotting
- Chapter 2: Manipulating time variables, grouping and summarising
- Chapter 3: Loops, if statements and simulation
- Chapter 4: Manipulating string variables
- Chapter 5: Web scraping, interactive graphics, Shiny apps (optional)

Hi, I'm Stefan

- Write me at **s.muehlbauer@lancaster.ac.uk** (much preferred)
- Or message me on MS Teams

Coursework

- 25% R coursework
 - Release Date: 26.10.2026
 - Due date: 20.11.2026
- 25% Python coursework
 - Release Date: 16.11.2026
 - Due date: 11.12.2026
- 50% Final project, using both R and Python
 - Release Date: 23.11.2026
 - Due date: **29.01.2027 (!)**

Set-up

R ≠ RStudio

R:

- The programming language
- The basic program for writing code in R

RStudio:

- An integrated development environment (IDE)
- Many helpful features:
 - Shiny apps
 - Creating your own R package
 - **R Markdown** files: compile to PDF, HTML, slides, etc.
 - Integration with e.g. Git and GitHub

In a university computer lab

- R and RStudio are available through AppsAnywhere
- Point Microsoft Edge (works better than Chrome) at <https://apps.lancaster.ac.uk/>
- Select *Launch* on RStudio (includes R, RTools, Java and most packages required for this module)
- RStudio is added to your programs in the Cloudpaging Player and will (probably) launch
- If it doesn't launch automatically, double-click it in the Cloudpaging Player
- **Supported by IT services**

On your personal laptop: local installation

Note: neither IT services nor I can provide support for this. Use it at your own risk.

1. Download R from CRAN: <https://cran.r-project.org/> or <https://www.r-project.org/>
 - Official R repositories (also for packages)
2. Download RStudio Desktop (free) from <https://www.rstudio.com/>
3. Alternatively, use Posit Cloud: <https://posit.cloud/> (free account needed)

Intro to R (finally!)

Today

- What is R?
- Tips on coding
- Hands-on Chapter 0
 - Basic calculations
 - Basic plots
 - Using functions
 - Creating R scripts

What is R?

- A programming language and software environment for statistical computing and graphics
- Widely used among statisticians and data scientists
- Great for developing statistical software and data analysis
 - A wide variety of statistical and ML techniques
 - A comprehensive set of tools for data manipulation, calculation and graphical representation

My advice on coding

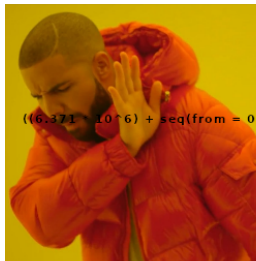
- Programming is not a spectator sport
 - **Learn by doing**
- Annotate your code
 - You write code for someone else, e.g. your future self
- DIE: Duplication Is Evil
 - Avoid repeated lines of code
 - Use functions (you will learn how to write them)
- Human brains remember patterns: pick a coding style you like and stick to it

My advice on coding

- Avoid long horizontal lines of code, think vertically
 - RStudio shows a guideline you should never cross
- Break code into smaller sections rather than long paragraphs
- Name functions and variables carefully
- Indent your code to make it more readable

My advice on coding

- Style and readability do matter



```
((6.371 * 10^6) + seq(from = 0, to = 35880 * 1000, by = 1000))^2
```



```
r_earth <- 6.371 * 10^6  
dist <- seq(from = 0,  
            to = 35880 * 1000,  
            by = 1000)  
r <- r_earth + dist  
r^2
```

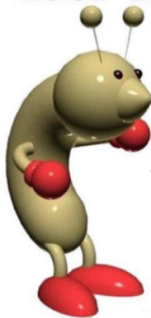
The internet (and coding)

- The internet is (usually) your friend when you're stuck
 - R help files, e.g. `help("lm")`, `?lm` or `?"*"`
 - You can often search for the solution
 - Online forums, e.g. <https://stackoverflow.com>

The internet (and coding)

- Be wary of generative models like ChatGPT. **Do not take their output for granted.**
- University policy on acknowledging AI in assessed work

Cool bug Fact's



we are aware of
the existence of
chat-GPT and GPT-4

This content was generated by ChatGPT-4. It may contain errors or inaccuracies. Please verify the information before using it.

To help you learn and remember code

- There are many cheat sheets on the web
- RStudio cheat sheets: <https://www.rstudio.com/resources/cheatsheets/>
- I uploaded some of them to Moodle

A typical R script file

```
# Description of what I am doing
# Date

# Setup ----
# Load in required packages
library(tidyverse)

# Do some calculations
x <- 1234 + 4321
x ^ 2

# etc!
...
```

Time to practise coding yourself

- Try to run R and RStudio
- Work through Chapter 0 of the notes
- Start with the exercises in the notes:
 - Section 0.3.5
 - Section 0.6.3
 - Section 0.7.4

R Markdown & Reproducibility

Reproducible research

- For every analysis, aim for a script that reads in the raw data, prepares the variables, performs the analysis and produces the plots
- You should be able to rerun it from beginning to end without bugs
- This lets you prove to others, and more importantly to yourself, what you have done
- Ideally, also save the output file (.Rout or knitr report) generated from your script
- Do not just type commands at the prompt and copy the output

The traditional workflow

1. Write code in an R script
2. Run code to get results, i.e. numbers and plots
3. Copy the results somewhere else, e.g. MS Word or $\text{\LaTeX}$
4. Insert the plots manually into the report
5. Find something wrong, back to the R script
6. Forget to save (some of) the updated results
7. The report contains some old and some new results
8. Others cannot reproduce your analysis

The reproducible way

- Ideally, we have a “notebook” file that contains
 - chunks of code
 - any accompanying text
- The report is generated dynamically: when the code or data change, the results change accordingly
- The generated document interweaves
 - the results, i.e. numbers and plots
 - the text explaining these results
 - any code required

The reproducible way

- This is the notebook approach:
 - In R: R Markdown
 - In Python: Jupyter notebooks
- This is how you will write your coursework
 - Write everything in **one** R Markdown file
 - Generate an HTML document
 - Submit the script file **and** the generated document
 - Easy to do in RStudio (instructions on the following slides)

R Markdown notebooks

- In RStudio click: File | New File | R Notebook
- RStudio generates a basic R Markdown file that looks like this:

```
---  
title: "R Notebook"  
output: html_notebook  
---
```

- The file extension is .Rmd, not .R. This means
 - you have to write R code in designated areas, but
 - you can write normal text elsewhere.
- The lecture notes were made with R Markdown
- Try following the instructions in this file; I will demonstrate some of them

R Markdown notebooks

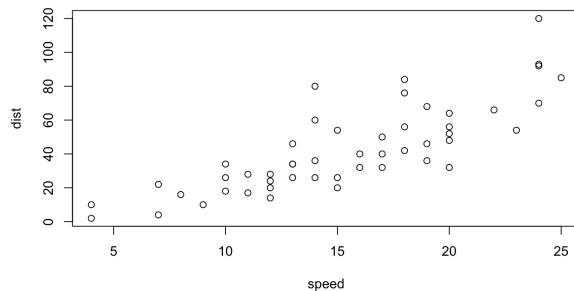
- *Execute a chunk by clicking the Run button within the chunk, or place your cursor inside it and press Ctrl+Shift+Enter.*

```
```{r}  
plot(cars)
```
```

R Markdown notebooks

This prints both the chunk and the result:

```
plot(cars)
```



R Markdown notebooks

- *Add a new chunk by clicking the Insert Chunk button on the toolbar or pressing Ctrl+Alt+I.*
- Type the R code for your analysis within the chunk
- Execute the new chunk as before

R Markdown notebooks

- *When you save the notebook, an HTML file with the code and output is saved alongside it (click Preview or press Ctrl+Shift+K to preview it).*
- This HTML file is only a preview and is **not** the generated document

R Markdown notebooks

- To generate the actual document, click the black arrow next to *Preview* (or *Knit* when you reopen the file)
- Choose *Knit to HTML* or *Knit to PDF*. The document pops up **and** is saved automatically in your working directory
- Knitting to PDF requires a $\text{\LaTeX}$ installation on your system

Writing coursework

1. Create a new R Markdown document
2. Edit the title, e.g. `title: R Coursework 0`
3. Add an author, e.g. `author: Student ID #####`
4. Set output: `html_document`
5. Save the file (File | Save) with the `.Rmd` extension
6. Remove the example text and code chunks

Writing coursework (cont'd)

1. Repeat for each part/question:

- Put your R code in a new chunk, i.e. between ``{r}`` and ```
- Click the green triangle to run a single chunk **or** the black arrow next to *Run* to run multiple chunks
- Check whether the results are correct and make sense
- Add text to explain your results where necessary

2. Click the black arrow next to *Knit* and generate an HTML or PDF

- If you specified output: `..._document`, click *Knit* to compile directly

Summary & final remarks

- We have learned how to
 - write R code in RStudio
 - generate a simple yet reproducible document
- This will go a long way in this course
- The coursework format varies, but the workflow and submission are the same
- The learning curve might not be gentle, but
 - this is the industry standard
 - things will get easier later this year
- Spend some time on the notes and these slides

Backup: Exercise solutions

For those not in LZSCC:

- This slide deck is also available at <https://smba.github.io/r-programming/intro>
- The solutions to the exercises from this session will be made available there as well after October 21, 2026.